

DFX 논문 리뷰(1)

25/12/24 세미나

목차

- 1. Abstract
- 2. Introduction
- 3. Background
- 4. Motivation
- 5. DFX Architecture

Abstract

Transfomer?

- 트랜스포머는 데이터 센터의 자연어 처리(NLP) 서비스에서 널리 사용되는 딥러닝 언어 모델, 이 중 Generative Pre-trained Transformer(GPT)는 텍스트 생성 또는 자연어 생성(NLG) 분야에서 뛰어난 성능을 달성
- 대규모 입력을 처리하는 요약 단계(summarization stage), 한번에 한 단어씩 생성하는 생성 단계(generation stage)로 이어짐.

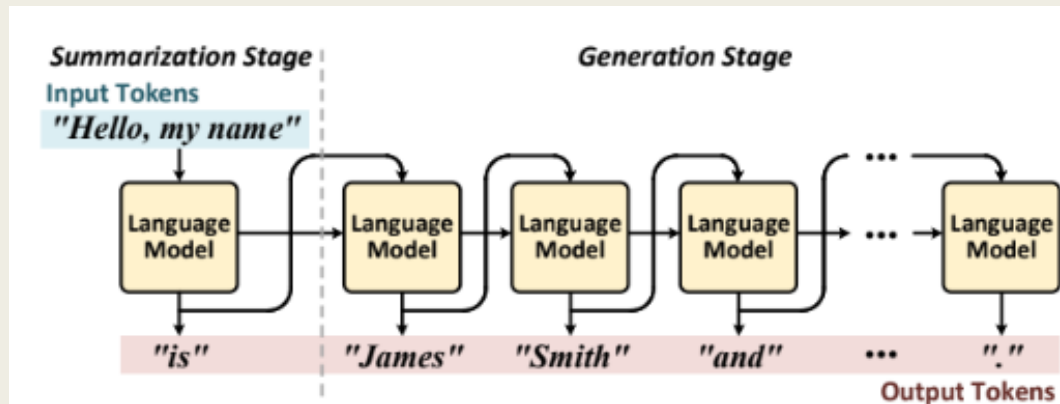
GPU의 쓰임 문제와 해결 방안

- GPU는 대규모 입력을 병렬로 처리하는데 특화, 요약 단계에서는 특화 but 생성 단계에서는 성능 크게 저하 -> 높은 지연 시간 해결을 위한 하드웨어 플랫폼 필요
- 본 논문에서는 요약 및 생성 단계 모두에서 낮은 지연 시간과 높은 처리량을 바탕으로 GPT-2 모델 추론을 end-to-end로 실행하는 multi-FPGA acceleration appliance인 DFX를 제안
- DFX는 최신 GPT-2 모델에서 4개의 NVIDIA V100 GPU 대비 5.58배의 속도 향상과 3.99배의 에너지 효율성을 달성했습니다. 또한 DFX는 GPU appliance보다 비용 효율성이 8.21배 더 높음

Introduction

Transformer/GPT

- 트랜스포머는 데이터 센터의 자연어 처리(NLP) 서비스에서 널리 사용되는 딥러닝 언어 모델, 순환 신경망(RNN) 과 장단기 메모리(LSTM) 의 고질적인 문제였던 재귀 연산 및 전역 의존성 결여 문제를 해결함
- 이 중 Generative Pre-trained Transformer(GPT)는 텍스트 생성 또는 자연어 생성(NLG) 분야에서 뛰어난 성능을 달성
- 대규모 입력을 처리하는 요약 단계(summarization stage), 한번에 한 단어씩 생성하는 생성 단계(generation stage)로 이어짐.
- GPU의 대규모 병렬 연산 유닛은 요약 단계에서는 높은 성능을 발휘, 그러나 생성 단계에서는 GPU가 순차적 처리에 적합하지 않아 연산 유닛 활용도(utilization)가 심각하게 떨어지며 성능 저하가 발생

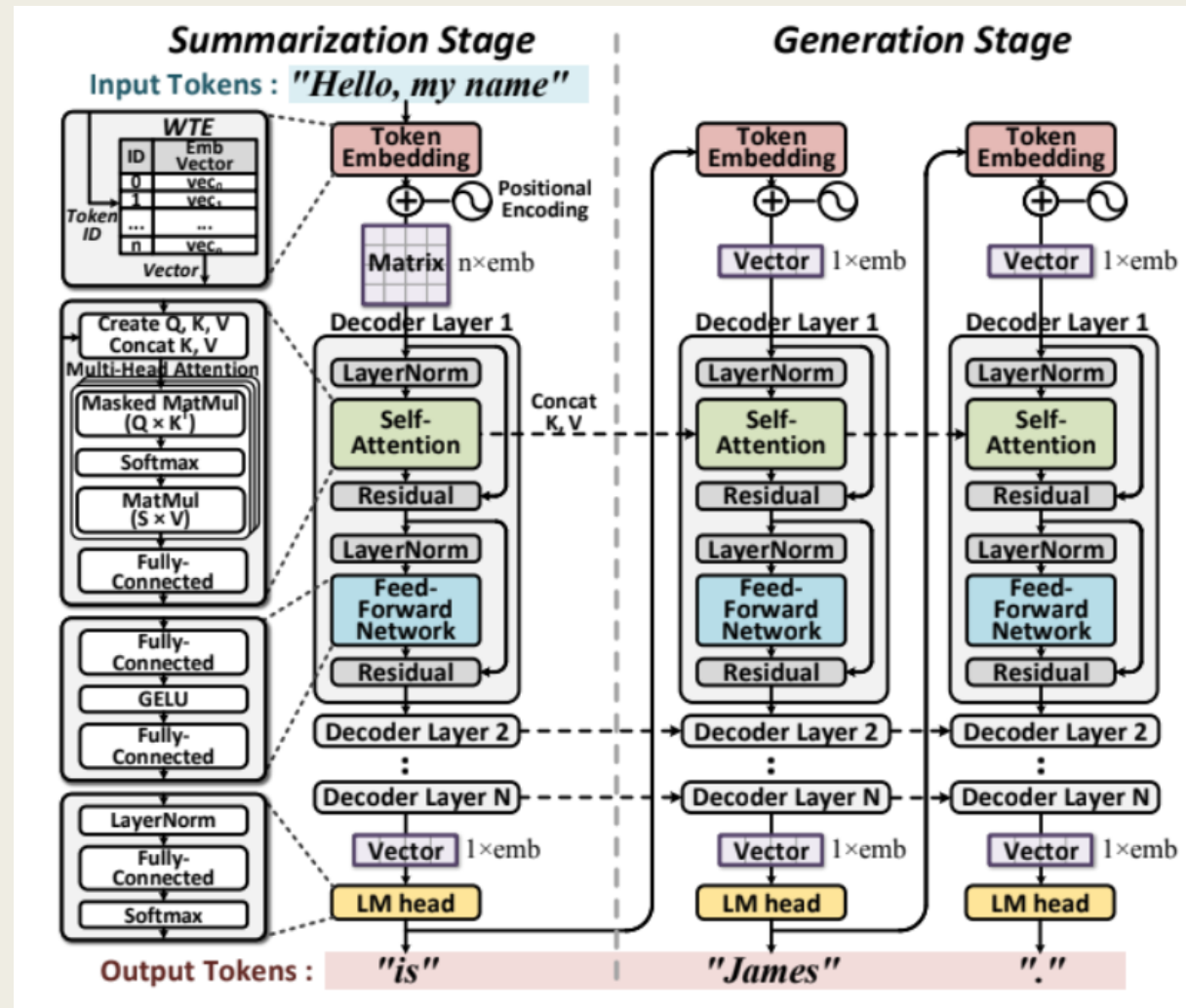


멀티 FPGA 가속 appliance DFX

- 이전: 문맥 이해를 위한 행렬 곱셈과 softmax로 구성된 attention 메커니즘이 관심사
- 전체 GPT 연산을 처음부터 끝까지(end-to-end) 지원할 수 있는 통합된 프로그래밍 가능 아키텍처가 필수적 -> DFX 제안
- 모델 크기 증가에 대응하기 위해 DFX는 멀티 디바이스 시스템에서 모델 병렬화를 사용하여 병렬로 작동하는 물리적 연산 코어의 수를 늘리는 동시에, 각 디바이스에 전체 작업 부하를 균등하게 할당 + 기존의 ASIC 대신 FPGA를 활용

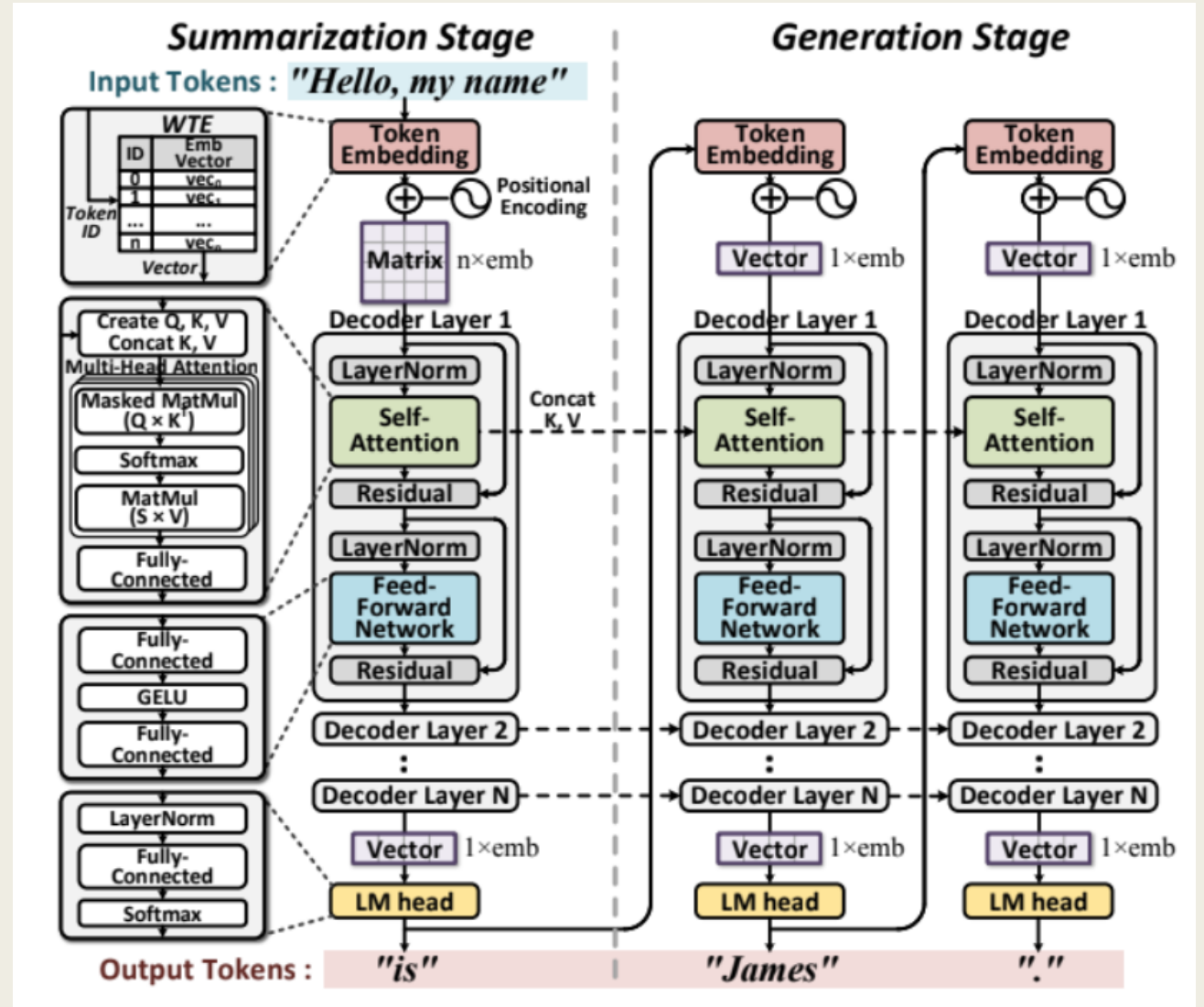
Background

GPT 구조



디코더 레이어 연산

- 1. Token Embedding
- 2. Self-Attention
- 3. Feed-Forward Network
- 4. LM head



토큰 생성 단계

- 1. 요약 단계
- 2. 생성 단계

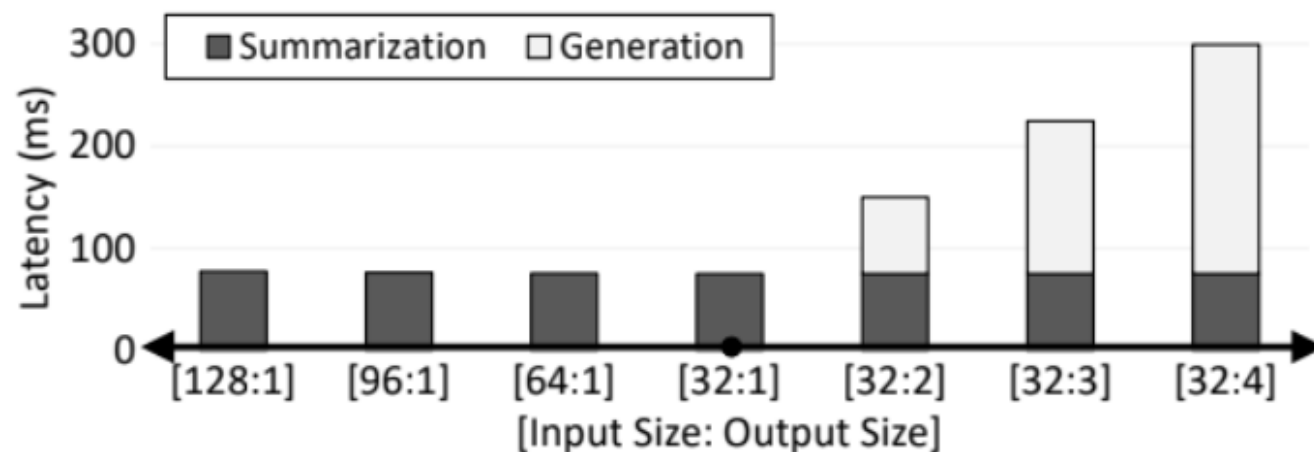
Parallelism in Deep Learning

- 1. Data Parallelism: 병렬화는 여러 개의 연산 장치(GPU, FPGA 등)에 동일한 모델을 복제해 두고, 입력 데이터(배치)를 나누어 처리하는 방식(학습에 적합, 추론에는 x)
- 2. Model Parallelism: 모델의 파라미터 자체를 여러 장치에 분산시키는 방식
 - 2-1. pipelined parallelism: 모델의 레이어를 그룹으로 묶어 장치별로 할당
 - 2-2. intra-layer parallelism: 하나의 레이어 안에 있는 거대한 행렬 곱셈 연산을 쪼개어 여러 장치가 동시에 수행하게 함
→ DFX가 채택한 방식

Motivation

Sequential Characteristic

- GPU는 대규모 병렬 연산 유닛과 높은 메모리 대역폭을 갖추고 있어 요약 단계에서 대규모 입력 토큰을 처리하는 데 이상적
- 생성 단계의 순차적 프로세스는 병렬화가 불가능하며, 연산 집약도가 낮아 GPU의 거대한 연산 유닛들을 효과적으로 활용하지 못함
→ 심각한 underutilization 발생
- 지연 시간이 배치 크기에 따라 증가



End-to-End Acceleration

- 이전 연구들은 트랜스포머의 attention 메커니즘만 가속하는 것을 목표
- 이러한 아키텍처를 기존 애플리케이션이나 서비스에서 end-to-end로 실행하려면 나머지 프로세스는 CPU에서 완료되어야 함
- Domain-specific accelerators는 이러한 복잡한 계산에 특화된 하드웨어 설계가 가능

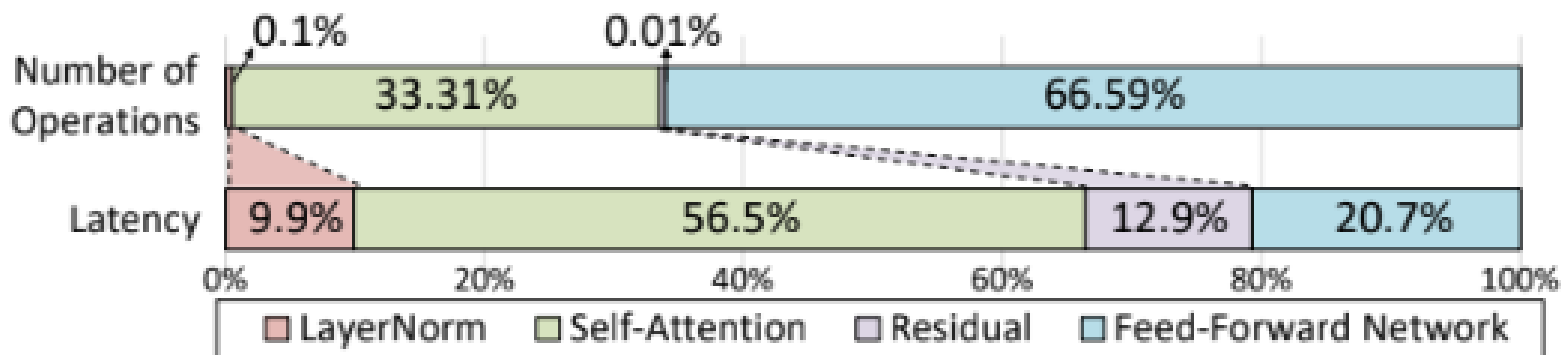


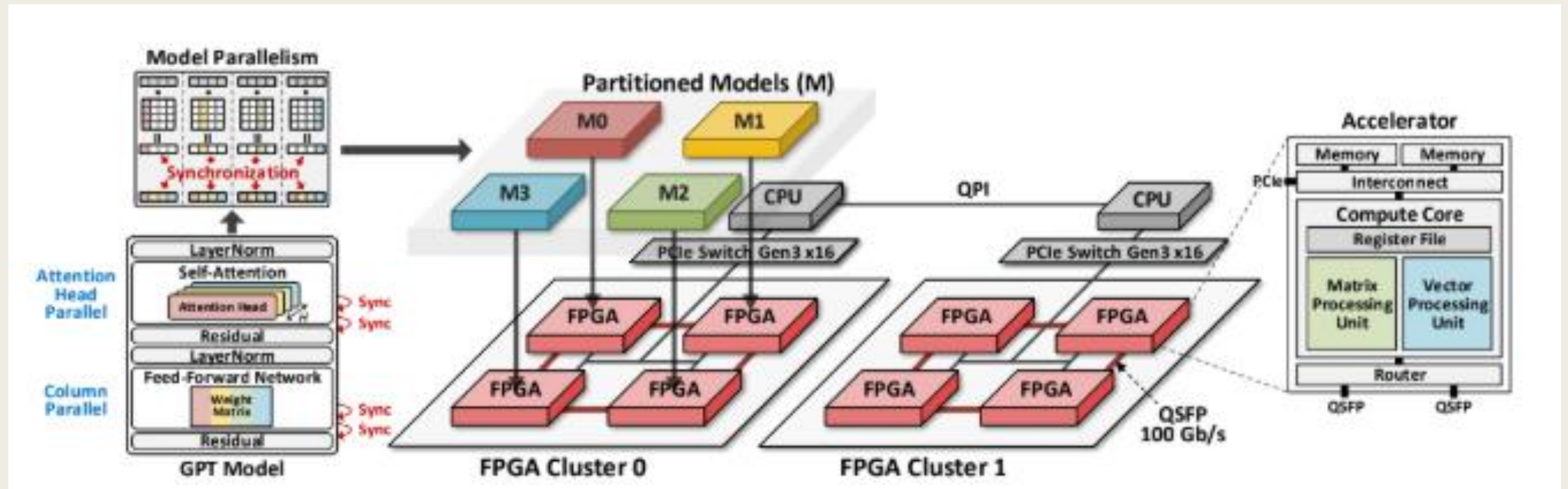
Figure 4. GPT-2 latency and number of operations breakdown on GPU.

Parallel Computing

- GPT-2 모델의 차원이 커짐에 따라 단일 장치는 필요한 계산을 수행하기 위한 메모리 대역폭과 용량이 모두 부족하여 충분하지 않음 -> 여러 장치가 워크로드를 공유해야 함
- 추가적인 지연 시간을 최소화하면서 병렬 연산량을 최대화할 수 있도록 모델 병렬화와 효율적인 네트워크를 채택한 멀티 디바이스 시스템이 필요

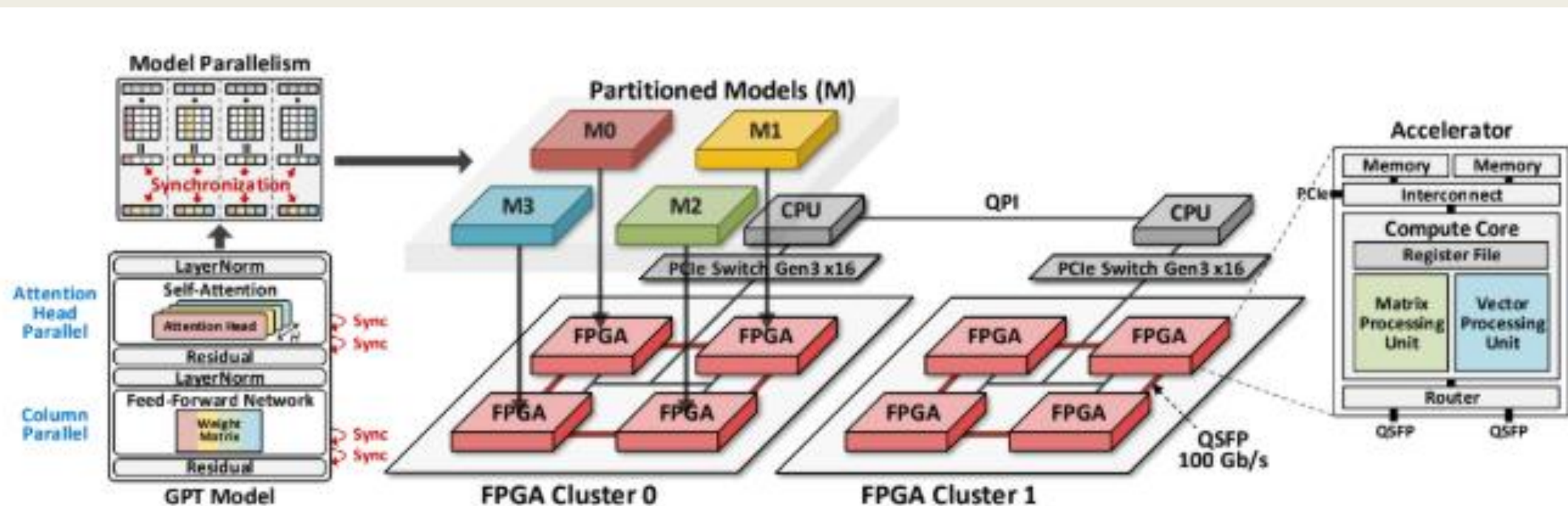
DFX Architecture

Architecture Overview



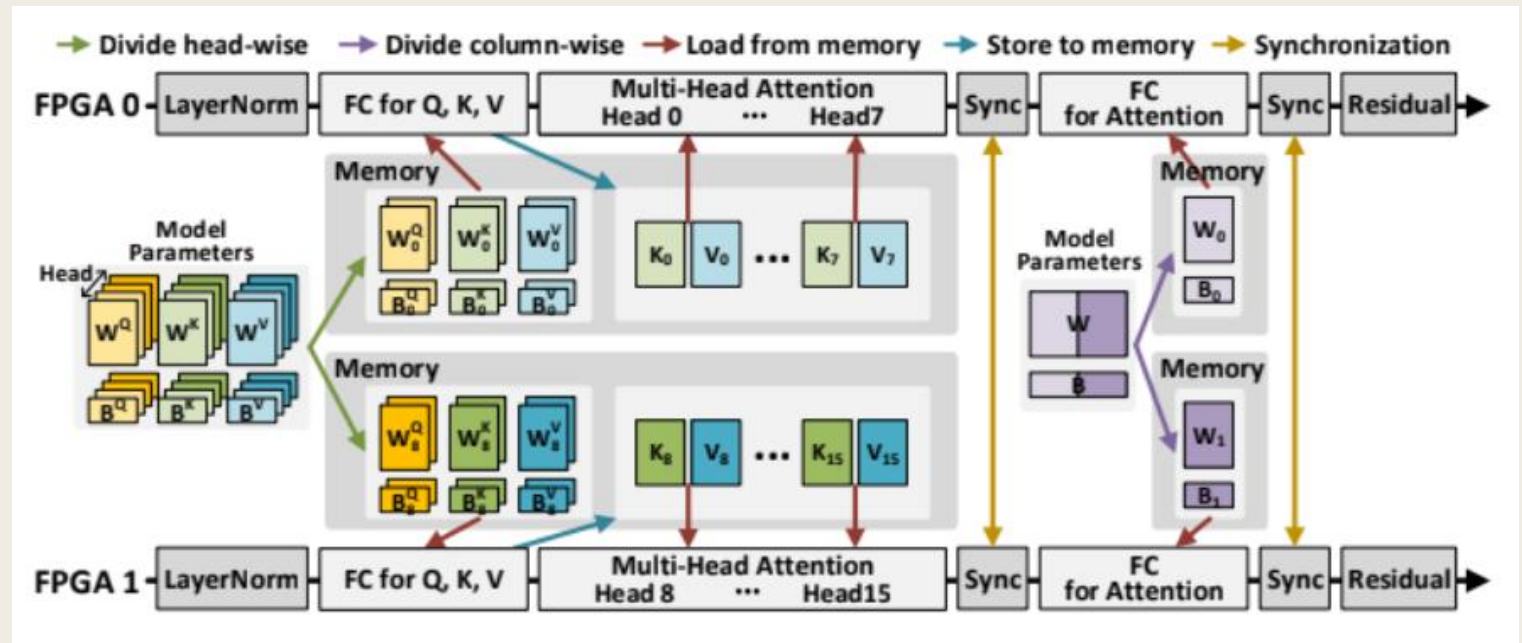
Architecture Overview

- 듀얼 소켓 CPU와 다수의 FPGA로 구성됨
- 하나의 CPU와 동일한 FPGA 클러스터가 하나의 시스템 형성
- FPGA는 16GB/s 속도로 데이터를 전송하는 PCIe Gen 3 서브시스템을 통해 호스트 CPU와 연결
- 각 FPGA는 두 개의 QSFP 포트에 제한되어 있음



Homogeneous Multi-FPGA Cluster

- 동기화 오버헤드를 최소화, 작업자 수에 비례하여 행렬 연산의 지연 시간 줄이기 위해 intra-layer parallelism 방식 채택
- 데이터 동기화
각 FPGA가 자기 조각을 옆으로 밀어주고, 받은 조각을 다시 전달하는 과정을 반복 -> 전체 벡터 똑같아지게 만들 -> 4번의 동기화 수행



Memory Mapping

- FPGA는 각각 이론적 최대 대역폭이 460GB/s인 8GB HBM과 38GB/s인 32GB DDR을 활용
- 가중치 행렬은 HBM, 입력/출력 토큰, 바이어스(bias) 벡터 및 기타 모델 파라미터는 DDR에 저장
- 표준 반정밀도 부동 소수점(FP16) 모델 파라미터를 사용

Instruction Set Architecture

- DFX ISA는 3가지 유형이 있다.
1. Compute 2. DMA 3. Router
- Compute 명령어: 주요 처리 유닛을 구동하기 위한 것
(type, src1, src2, dst) 형식을 가짐
- DMA, Router 명령어: DMA, 네트워크 라우터를 제어하여 코어와 데이터 주고 받음
(type, src, dst, xfer_size) 형식

Compute 명령어

■ 1. Matrix instructions

- 1) Conv1D: Query, Key, Value 행렬 생성과 피드 포워드 네트워크에서 사용
- 2) MaskedMM: 어텐션의 스코어 계산
- 3) MM: 마스킹이 없는 MaskedMM과 동일

■ 2. Vector instructions

- 1) LayerNorm
- 2) Softmax

Instruction Set Architecture

Input: in_emb , input embedding vector

Output: out_emb , output embedding vector

Parameter: H , number of attention heads

```
1: /* Layer Norm */
2:  $lnorm1 = LayerNorm(in\_emb, \gamma_1, \beta_{l1})$ 
3: /* Self-Attention */
4:  $value = Conv1D(lnorm1, W_v, b_v)$ 
5:  $key = Conv1D(lnorm1, W_k, b_k)$ 
6:  $query = Conv1D(lnorm1, W_q, b_q)$ 
7: for  $h = 0$  to  $H$  do
8:    $mat = MaskedMM(query[h], key^T[h])$ 
9:    $redu\_max = ReduMax(mat)$ 
10:   $score = Softmax(mat - redu\_max)$ 
11:   $attn'[h] = MM(score, value[h])$ 
12: end for
13:  $attn = Sync(attn')$ 
14:  $c\_attn' = Conv1D(attn, W_a, b_a)$ 
15:  $c\_attn = Sync(c\_attn')$ 
16: /* Residual */
17:  $c\_attn = c\_attn + in\_emb$ 
18: /* Layer Norm */
19:  $lnorm2 = LayerNorm(c\_attn, \gamma_2, \beta_{l2})$ 
20: /* Feed-Forward Network */
21:  $ffn1' = GELU(Conv1D(lnorm2, W_{f1}, b_{f1}))$ 
22:  $ffn1 = Sync(ffn1')$ 
23:  $ffn2' = Conv1D(ffn1, W_{f2}, b_{f2})$ 
24:  $ffn2 = Sync(ffn2')$ 
25: /* Residual */
26:  $out\_emb = ffn2 + c\_attn$ 
```